

Projeto 1

Fanorona com sockets TCP

Relatório de implementação e guia de apresentação

Instituto Federal do Ceará
Engenharia de Computação
Programação Paralela e Distribuída - 2026.2
Professor Cidcley T. de Souza

Objetivo e resultado

O projeto implementa uma partida de Fanorona entre dois clientes Java Swing conectados a um servidor TCP. O servidor mantém o tabuleiro oficial, valida as regras e transmite o mesmo estado aos dois jogadores. A entrega reúne código comentado, testes, instruções de execução e dois arquivos JAR, um para o servidor e outro para o cliente.

A base é o projeto de Dara do semestre 2026.1. Foram preservados o modelo cliente-servidor, a organização Maven e os pacotes de responsabilidade. O motor do jogo e a interface foram adaptados ao Fanorona. A pasta de entrega é PPD_2026.2/sockets.

Requisito do enunciado	Implementação
Controle de turno e início	P1, primeiro identificado no lobby, joga com brancas e inicia.
Movimentação e capturas	Tabuleiro 5 × 9, destinos legais destacados e escolha entre aproximação e afastamento.
Desistência e vencedor	Resultado do servidor, com motivo e estatísticas; desconexão também encerra a partida.
Chat durante a partida	Mensagens UTF-8 entre os participantes, inclusive fora do próprio turno.
Código e executável	Fontes em src, build.ps1, pom.xml e JARs em target.

Leitura sugerida: arquitetura na página 2; regras na página 3; modificações na página 4; execução na página 5; testes e apresentação na página 6.

Arquitetura e programação distribuída

Cada cliente apresenta um tabuleiro, mas somente o servidor decide se uma ação é válida. O cliente envia uma intenção; a sessão valida a identidade da conexão, a revisão do estado, o turno e as regras. Só depois publica o novo estado. Assim, os dois tabuleiros seguem a mesma autoridade.

Classe ou pacote	Responsabilidade
client/FanoronaClient	Conectar, enviar comandos em executor e receber dados em thread própria.
client/GameFrame e ResultsDialog	Interação Swing, desenho das linhas e peças, chat e resultado.
server/FanoronaServer	Aceitar conexões, manter espera e parear jogadores em sessões.
server/ClientHandler	Ler comandos de uma conexão e escrever respostas pela fila de saída.
server/GameSession	Serializar ações de uma partida, aplicar regras, publicar estado e resultado.
game/Board e Piece	Motor determinístico e identidade das peças; sem dependência de rede.
shared/Protocol	Enquadramento das mensagens, Base64, UTF-8 e limites de entrada.

Threads e exclusão mútua

O laço de accept não espera o nome do jogador. Cada handler executa a identificação com timeout de 15 segundos. O método sincronizado `GameSession.process` impede que duas threads alterem simultaneamente o mesmo tabuleiro. Sessões distintas têm monitores independentes e podem progredir em paralelo.

As mensagens são colocadas em filas de saída de até 256 entradas por cliente. Uma thread de escrita consome cada fila, preservando a ordem sem fazer a sessão aguardar a rede. Fila saturada fecha a conexão. O lobby aceita até 100 clientes. O fechamento por EOF e por exceção passa pelo mesmo tratamento de desconexão.

Protocolo e interface

TCP transporta bytes em ordem; as mensagens são delimitadas por fim de linha. Cada linha tem no máximo 8192 caracteres. Campos usam o separador `|`, e textos usam Base64 sobre UTF-8. Base64 apenas codifica o conteúdo. Não há autenticação nem TLS.

STATE contém revisão, turno, peça da sequência, 45 posições e alternativas legais. MOVE e END_TURN incluem a revisão vista pelo cliente. Na interface, `SwingUtilities.invokeLater` transfere as atualizações para a EDT; conexão, leitura e envio ficam fora dessa thread.

Regras e decisões de implementação

Posição inicial e deslocamento

O tabuleiro possui cinco linhas e nove colunas. P1 começa com 22 peças brancas, P2 com 22 pretas; o centro é a única interseção vazia. A peça se desloca para um ponto vizinho vazio ligado por uma linha. Diagonais existem somente em pontos cuja soma linha + coluna é par, usando índices a partir de zero.

Captura por aproximação e afastamento

Na aproximação, o deslocamento deixa a peça junto à linha inimiga além do destino. No afastamento, a peça sai de junto da linha inimiga atrás da origem. São retiradas todas as peças adversárias contíguas nessa direção, até uma borda, um ponto vazio ou uma peça aliada. Quando as duas formas são possíveis, o jogador escolhe apenas uma.

Sequência de capturas

Havendo captura disponível no começo do turno, é obrigatório capturar. Depois de uma captura, o motor oferece continuações apenas para a mesma peça. Um conjunto de posições visitadas inclui a origem e cada destino, impedindo revisitas. O último vetor de movimento impede repetir a direção imediatamente anterior. Sem continuação legal, o turno termina automaticamente.

O enunciado afirma que o jogador pode continuar a sequência. Por isso, a interface oferece Encerrar sequência após uma captura, mesmo que ainda existam continuações. Esse botão não permite passar um turno sem jogar.

Vitória e empate

Capturar todas as peças adversárias encerra a partida. Como convenção adicional, o jogador sem movimento legal no início de seu turno perde por imobilização. Desistência ou desconexão concede vitória ao adversário. Uma oferta de empate é aceita se o outro jogador também acionar Empate; uma jogada cancela a oferta pendente. Não há empate automático por repetição.

Relação com as fontes

O PDF do professor define os requisitos centrais e a continuação opcional. A referência complementar da ICGA descreve captura em linha, escolha entre modos, restrições de direção e de visita. As decisões adicionais ficam explícitas para permitir a discussão da variante durante a apresentação. Não há implementação da revanche Vela.

Exemplo didático: uma peça em A3 que se move para B3 por aproximação retira adversárias contíguas em C3, D3 e assim por diante; para no primeiro ponto vazio ou aliado. O destino B3 precisa estar vazio. No afastamento de C3 para D3, a captura começa em B3 e segue para A3.

Principais modificações em relação ao Dara

O reaproveitamento é arquitetural. A base forneceu a separação entre cliente, servidor, sessão, tabuleiro e protocolo; os mecanismos de Socket, ServerSocket, handlers e atualização Swing orientaram a implementação. As regras do Dara não foram mantidas no novo motor.

Aspecto	Semestre 2026.1	Semestre 2026.2
Tabuleiro	5 × 6, com fase de colocação.	5 × 9, com 44 peças na posição inicial.
Captura	Formação de trinca e retirada escolhida.	Aproximação ou afastamento de uma linha contígua.
Movimentos	Ortogonal e regras de alinhamento do Dara.	Linhas ortogonais e diagonais; captura e sequência.
Estado de rede	Eventos separados de colocação, movimento e remoção.	Snapshot integral com número de revisão e movimentos legais.
Identificação	Leitura do nome no laço de aceitação.	Handshake no handler, com timeout.
Encerramento	Desconexão tratada sobretudo por exceção.	Tratamento de EOF e exceção, com resultado único.
Demonstração	Comando /endgame no chat altera peças.	Cenários isolados em testes; chat nunca altera o tabuleiro.
Empacotamento	Maven, shade e dependência de JUnit.	JARs sem dependências e testes executáveis com o JDK.

Organização preservada

A raiz continua sendo sockets, contendo pom.xml, README.md, src/main/java, src/test/java e target. A hierarquia br/com/victorsfec permanece; dara foi substituído por fanorona. Os pacotes client, server, game e shared foram mantidos. A pasta docs acrescenta o relatório, o contrato do protocolo e o gerador deste PDF.

Diferenças operacionais

O servidor passa a operar em terminal, facilitando execução sem interface gráfica. Os logs diários em arquivo e a tentativa de reconexão automática da base não foram transportados. O chat permanece em memória na interface. Uma conexão perdida encerra a participação; uma nova partida exige abrir novos clientes.

Compilação e uso

Requisitos

Use Java 21 ou superior para executar e JDK 21 ou superior para compilar. Java 8 não é compatível com os executáveis. Swing e sockets são bibliotecas do próprio Java; não há bibliotecas adicionais para executar o jogo. A compilação validada usa o script PowerShell com o JDK portátil.

Gerar executáveis e testar

Abra um terminal em PPD_2026.2/sockets e informe a pasta do seu JDK. O script compila fontes, compila testes, executa a suíte e empacota os dois JARs somente se os testes passarem.

```
.\build.ps1 -JdkHome "C:\caminho\jdk-21"
```

A alternativa Maven é `mvn clean package`. O pom mantém a organização do projeto anterior e executa AppTest na fase test. O primeiro uso do Maven requer download dos plugins; essa alternativa não substitui a validação realizada pelo script.

Abrir uma partida local

Execute o primeiro comando em um terminal e o segundo em outros dois terminais. Nos clientes, escolha nomes, host localhost e porta 12345. O primeiro identificado é P1 e inicia a partida.

```
java -jar target/sockets-1.0-server.jar  
java -jar target/sockets-1.0-client.jar
```

Se o comando java da máquina apontar para Java 8, use o executável do JDK 21 pelo caminho completo. O README registra o caminho do JDK portátil usado nesta entrega. Uma porta diferente pode ser passada ao servidor como argumento, por exemplo 15000.

Executar em máquinas diferentes

Inicie o servidor, consulte seu IPv4 com `ipconfig` e informe esse endereço nos clientes. Use a mesma porta em todas as telas. A máquina servidora precisa aceitar conexões TCP nessa porta pelo firewall. Os clientes se conectam ao servidor, não diretamente entre si. Localhost sempre designa a própria máquina.

Operação da interface

Selecione uma peça própria para visualizar os destinos verdes; clique no destino para enviar a jogada. Uma janela pede o tipo de captura se houver duas opções. Encerrar sequência só é habilitado quando a regra permite. Desistir e Empate pedem confirmação. A tela exibe vencedor e estatísticas ao fim e mantém o chat aberto.

Limites conhecidos

A verificação de rede foi local, por loopback TCP. A execução em duas máquinas físicas deve ser ensaiada na rede de apresentação. Não há persistência de partidas, autenticação, criptografia, retomada por reconexão ou heartbeat de aplicação; quedas silenciosas podem depender da detecção pelo TCP.

Verificação e roteiro de apresentação

Evidências de teste

A suíte AppTest usa verificações explícitas e lança `AssertionError` em caso de falha. Não depende de habilitar assertions da JVM nem de JUnit. Os testes de rede abrem um servidor em porta efêmera e clientes Socket reais com timeout de leitura. O resultado final da execução é registrado no arquivo `docs/TESTES.md`.

Grupo	Cobertura
Regras	Posição inicial, conexões, turno, limites, capturas A/W, escolha exclusiva, obrigatoriedade, sequência, direção, revisita, encerramento e vitória.
Invariantes	Partidas com sementes fixas verificam que movimento não cria peças e que a captura reduz apenas o adversário.
TCP	Identificação sem bloquear accept, pareamento, snapshots iguais, rejeição de comandos, chat, revisão antiga e resultado.
Concorrência e término	Sessões isoladas, desistência, EOF, empate e encerramento do laço de aceitação.
Interface	Renderização da própria tela Swing para revisão de disposição e legibilidade. O ensaio interativo continua sendo parte da preparação.

Demonstração sugerida

1. Inicie o servidor e conecte dois clientes. Identifique P1, P2 e o turno inicial.
2. Envie uma mensagem com acentos e mova uma peça para o centro.
3. Mostre a captura nas duas telas e a escolha A/W quando disponível.
4. Explique o botão de encerramento e a restrição à mesma peça na sequência.
5. Demonstre desistência, vencedor e estatísticas.
6. Abra outra dupla e feche um cliente para demonstrar a desconexão.
7. Execute a suíte para mostrar capturas, sequências e isolamento em cenários controlados.

Pontos para explicar no código

Comece em `Board.legalMoves` e `Board.move` para as regras. Depois acompanhe `ClientHandler.run` até `GameSession.process` para explicar como o comando atravessa a rede. Mostre o `synchronized` da sessão, as filas de escrita e o `STATE` com revisão. Termine em `FanoronaClient.listen` e `GameFrame.receive` para explicar a transferência de atualizações à EDT.

Fontes e materiais da entrega

Projeto 1.pdf, enunciado fornecido pelo professor; fontes de `PPD_2026.1/sockets`, base local do Dara; ICGA, Fanorona, disponível em <https://icga.org/icga/games/Fanorona/> (consulta em 06/09/2026). O README detalha execução e decisões; `docs/PROTOCOLO.md` descreve todos os campos das mensagens.